



**Dementias
Platform^{UK}**

Automated pre-processing pipeline for multimodal tabular health data

Saad Kiyani

Swansea University · developed and tested within the DPUK Data Portal

numerical

📁 categorical

📅 date

🕒 time

☰ multi-option

T free text

Cohort data arrives messy, and everyone cleans it differently

One table can hold numbers, categories, dates, clock times, multi-select answers and free text: "multimodal tabular".



Many kinds of data in one file

Each kind needs its own cleaning rules; a single script rarely gets all six right.



The errors are silent

A -99 that slips through becomes a real age of minus ninety-nine. Nothing crashes.



Two analysts, two datasets

Hand-written cleaning is done once, by one person, and cannot be repeated or audited.

subject_id	age	sex	assessment_date	languages
S001	62	m	12/03/2019	English, French
S002	-99	Female	Nov-18	english/French
S003	71	1	NA	eng
S006	999	male	Sep, 2017	French
S007	37	not recorded	30/01/2021	British English

Highlighted: values a hand-written script silently gets wrong. Coded missing, implausible, inconsistent labels, mixed date/time formats, aliases.

A configuration-driven approach



One file per cohort

A plain-text YAML file says what every column is and how it should be cleaned.



Same input, same output

Same config plus same extract gives the same table, every time, on any machine.

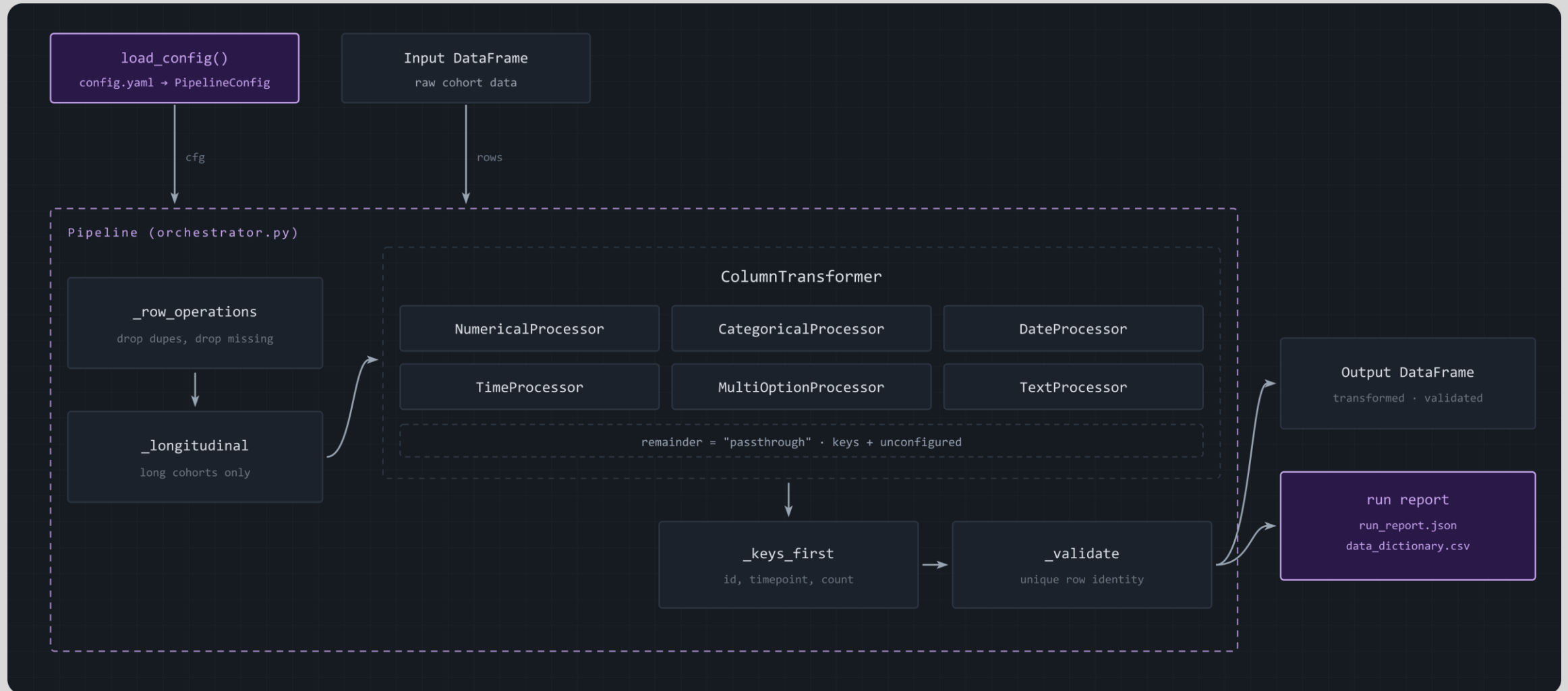


Decisions written down

Define semantic rules like aliases, ranges, missing codes in the file, not the script.

A researcher can read and correct the file without touching code. The config is the contract; the code is replaceable behind it.

How it works: from raw extract to cleaned table and report



Bad cleaning is usually silent. This pipeline is built to be loud.

Four rules that generate most of the design.



Never invents data

Dates and times are never imputed. No fuzzy label matching: aliases are exact and written down by the researcher. In a long cohort, no visit is fabricated and no trait is filled backwards.



Output shape comes from config, not data

The same YAML yields the same columns on every extract; a long cohort is never pivoted to wide. Where a width must be learned from data, the run warns and names the config field that would fix it.



Config mistakes fail at load

Misspell delimiter and the run refuses to start rather than splitting on the wrong character. 30 cross-field guards. A declared field that nothing reads is an error, not a no-op.



No silent skips

A configured column missing from the extract, an unrecognised validation key, a row operation that could not run: each is a recorded warning on the run, not a quiet pass.

Every run leaves an audit trail

You get the cleaned data and a record of exactly what happened to it: `run_report.json` + `data_dictionary.csv`

CONFIG SHA256

9195343f...3c3fb

hash of the parsed config: a comment edit leaves it alone, a behaviour edit moves it

PIPELINE CODE

git 9e5414d

which version of the processors produced this output

VERSIONS

pandas 3.0.3 · sklearn 1.9.0

Python 3.14.5; library versions recorded, never assumed

ROWS

9 in → 8 out

removed: 1 exact duplicate, 0 missing an id

output_column	completeness	meaning	missing representation	fitted parameters
age	100.0%	numeric value, cleaned	imputed with 62 (median); no missing values remain	{"fill": 62.0}
sex_Male	100.0%	indicator: sex == 'Male'. Source labels were alias-mapped first (6 replacements from mapping)	all-zero row across the block; indistinguishable from a value outside the vocabulary	vocabulary: declared
education_level	87.5%	ordinal rank of education_level; 0 = 'none'	NaN: missing, or a label outside the declared codebook	order: none 0 ... postgraduate 4
assessment_date	87.5%	parsed date; four declared formats. A month-only format fabricates day 1, so the day is not evidence	NaT; never imputed, because imputing a date invents an event that did not happen	none
languages_English	100.0%	indicator: languages contains 'English' after alias mapping	all-zero row across the block; indistinguishable from a value outside the vocabulary, and from none-selected	vocabulary: declared
symptom_notes_mentions_pain	87.5%	boolean: rule 'mentions_pain' matched	NaN for a missing cell; 0 means the rule genuinely did not match	keywords: pain, ache · bool

Rows taken from `data_dictionary.csv` after running the example config on a sample extract (some text abridged to fit).
Note the pairing: a completeness of 100% is only honest next to the sentence that says it was imputed.

The report never claims something that did not happen.

You do not write the config from scratch

The scaffolder drafts it from the data; a person reviews it; lint refuses to release it with a guess still in it.



Draft

```
python -m scaffold cohort.csv -o draft.yaml
```

Profiles the extract and infers a type per column. For every column it records the evidence it used: how much is missing, how many distinct values, the observed range, and any sentinel codes it spotted. Every guess is marked # REVIEW.

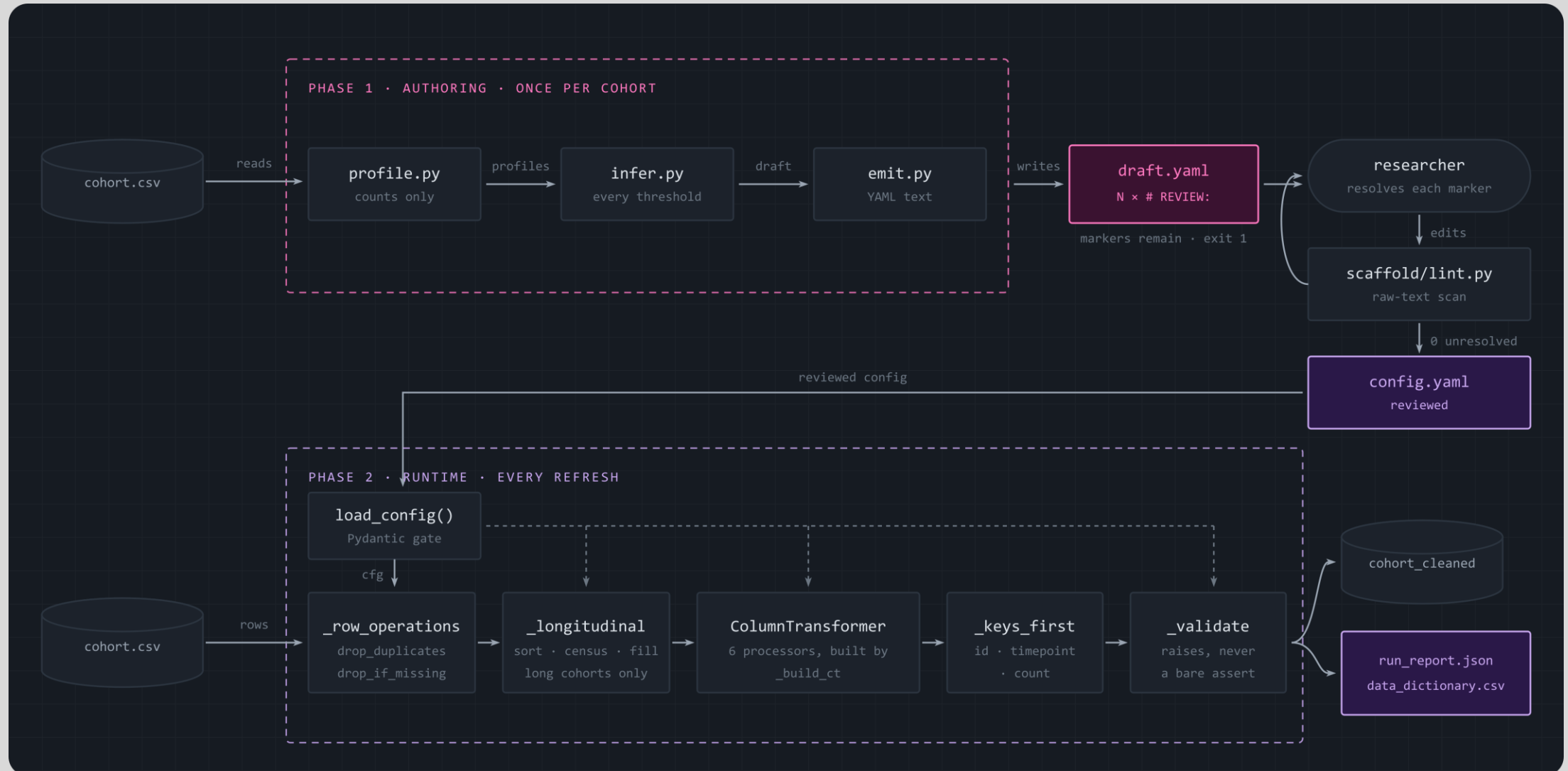


Review and lint

```
python -m scaffold.lint draft.yaml
```

Exit code 1 while any REVIEW marker or placeholder remains. Only a config with zero unresolved markers reaches the pipeline. Human in the loop by construction.

Putting it together: authoring and runtime



Tested on DPUK Data Portal cohorts

Wide and long cohorts, run inside the Portal.



Cam-CAN

[2676 rows × 821 columns]

Wide: one row per participant.



BRACE

[7574 rows × 187 columns]

Wide, and the long part of the cohort: many rows per participant, one per (participant, visit).



NEXT

Mapping back to the original layout

Every processor records what it did to its column; invert that record.

ML-ready format back to the original columns and values, so results can be read in the cohort's own terms.

Impact

Who benefits, and how.



Researchers

- Declare the cleaning once per cohort; rerun it on every extract
- Read and correct the rules without touching code
- Semantic choices are written down, so a collaborator can check them



The Data Portal

- Every run records config hash, code commit and library versions
- One data dictionary row per output column: what happened, and why
- Nothing invented, nothing silently skipped: warnings are recorded on the run



Work across cohorts

- One reviewed config per cohort, reusable across studies
- Output shape fixed by config, so downstream models survive a data refresh
- The scaffolder turns authoring into review



**Dementias
Platform^{UK}**

**From messy cohort extract to ML-ready table,
reproducibly, from one reviewed config.**

Questions

Saad Kiyani

Swansea University · DPUK Data Portal